

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Project Garuda: A Privacy-First Edge-AI Home Security System on Raspberry Pi 5 with Hailo-8L NPU

VEERAMANIKANTA GONUGONDLA¹

¹Department of Electronics and Communication Engineering (e-mail: manikantagonugondla@gmail.com)

Corresponding author: Veeramanikanta Gonugondla (e-mail: manikantagonugondla@gmail.com).

This work was carried out as an independent project. All hardware, software, and cloud services were self-funded.

ABSTRACT Nowadays, Home Security Systems are very expensive and depend heavily on the cloud to run AI models. Cloud Dependency involves increased expenses for infrastructure and is also privacy-invasive. We can also opt for open-source alternatives, which come with a trade-off in the integration between hardware and software due to compatibility of various hardware and software libraries. Most DIY setups either detect only motion with no second intelligence check or require a full server rack to run meaningful AI models. No widely available solution integrates real-time edge AI, remote access, voice control, and encrypted data handling on consumer hardware at a meaningful price point and with meaningful integration. Project Garuda is an attempt to close that gap. It is an AI-powered home security system built on a Raspberry Pi 5 integrated with a Hailo-8L neural processing unit — a 13 TOPS dedicated AI accelerator that handles deep learning inference without taxing the CPU. The system runs YOLOv8s object detection through a GStreamer pipeline managed by Hailo's TAPPAS framework, silently logging person detections and triggering alerts when it identifies dangerous objects — knives, scissors, or hammers — in frame. Alerts are dispatched as OTP-authenticated emails with a 60-second cooldown to prevent spam. The backend is a FastAPI web server, accessible remotely through a Cloudflare tunnel at api.veeramanikanta.in, secured with PBKDF2-SHA256 session authentication, a master key system, per-endpoint rate limiting, and strict CORS and security header policies. The frontend is a Progressive Web App. Video clips can be recorded, AES-256 encrypted, and sent over SSH to a remote host. Live video is available over both WebRTC and MJPEG. A voice assistant named Narada runs as a background thread, responding to a configurable wake word and routing commands either through rule-based matching or a Groq-hosted LLM — the same model backing the web UI's streaming chat interface. The system supports six operating modes — Do Not Disturb, email-off, idle, night, emergency, and privacy — schedulable automatically or set manually. Supporting subsystems include ARP-based LAN device presence tracking, a 180-second deadman watchdog that fires a tamper alert on missed heartbeats, a SQLite-backed offline event sync queue, and a multi-tier logging pipeline with rotation across persistent text and JSON files. The goal is a self-hosted, privacy-first security system that thinks at the edge, costs under US\$450–500 to build, and doesn't phone home to anyone.

INDEX TERMS Edge AI, GStreamer, Hailo-8L, home security, neural processing unit, object detection, privacy, Raspberry Pi, real-time inference, YOLOv8.

I. INTRODUCTION

HOME security systems have evolved from passive alarm triggers to intelligent platforms that can recognise specific objects, persons, and behaviours in real time. However, the commercial market remains bifurcated. On one end, cloud-dependent solutions from established vendors offload inference to remote data centres, introducing recurring subscription costs, network-latency penalties, and privacy con-

cerns because raw video leaves the premises [1]. On the other end, do-it-yourself (DIY) setups based on single-board computers typically offer motion detection at best, with no second-stage intelligence check and no integrated alert, voice, or encrypted-evidence pipeline [2].

Recent advances in dedicated neural processing units (NPUs) have begun to close this gap by bringing deep-learning inference to the edge at watt-level power budgets.

The Hailo-8L, for example, delivers 13 TOPS of INT8 compute over a single PCIe Gen3 lane and can be attached to a Raspberry Pi 5 through a standard M.2 HAT, giving a sub-US\$500 platform enough throughput to run YOLOv8-class detectors at more than 50 FPS without taxing the host CPU [3]. Bueno *et al.* [4] recently validated this exact hardware combination for real-time YOLOv8 inference in microscopy, confirming that the RPi 5 plus Hailo-8L pipeline can match GPU-accelerated systems in accuracy while consuming a fraction of the power.

A. RELATED WORK

Edge-deployed YOLO variants have been explored across several domains. Xu *et al.* [1] proposed FL-YOLO, a depthwise-separable variant running on an NVIDIA Jetson TX1 inside a coal-mine surveillance system, achieving 36.7 FPS at 76.7% AP through an edge-cloud cooperation framework. Al Amin *et al.* [5] implemented YOLOv3 Tiny on a Xilinx Kria KV260 FPGA, reaching 15 FPS at 99% accuracy for traffic-light classification at 3.5 W — demonstrating that dedicated accelerators can outperform general-purpose GPUs in power efficiency for fixed inference tasks. Dong *et al.* [6] introduced PG-YOLO, which replaces standard convolutions in YOLOv5s with Ghost modules and applies channel pruning plus knowledge distillation to achieve a 9× compression with only 0.1% accuracy loss, a technique directly applicable to NPU deployment. Chung *et al.* [7] proposed YOLO-LSD with CBAM attention for detecting small pedestrians at long range, and Cheng [8] presented RMN-YOLO with multi-scale edge enhancement for defect detection, both contributing architectural ideas for improving small-object recall on resource-constrained hardware. The Ultralytics YOLOv8 framework [9] provides the baseline architecture and training pipeline used in this work, and the Hailo TAPPAS framework [10] supplies the GStreamer elements that dispatch inference to the NPU.

Despite this body of work, no published system integrates all of the following on a single consumer-grade edge device: real-time multi-class object detection with a dedicated NPU, a cascaded classifier and depth-based anti-spoof stage, encrypted evidence capture, a voice assistant with LLM fallback, ARP-based presence monitoring, a tamper-detection watchdog, and a remotely accessible web server with session authentication and OTP two-factor verification.

B. CONTRIBUTIONS

This paper presents Project Garuda, an end-to-end AI home security system that runs entirely on a Raspberry Pi 5 with a Hailo-8L AI HAT+. The principal contributions are:

- 1) A custom-fine-tuned YOLOv8s detector for four security-relevant classes (Hammer, Knife, Person, Scissors) trained on a 4,982-image dataset and compiled to an INT8 HEF, achieving a test mAP@0.5 of 0.847 at 52.22 FPS on the Hailo-8L.
- 2) A three-stage cascade architecture (YOLOv8s + MobileNetV2 classifier + MiDaS depth estimator) that

adds weapon verification and photo/screen spoof rejection with less than 5 ms additional latency.

- 3) A twelve-engine software architecture — detection, alert, voice assistant, mode controller, presence monitor, web server, authentication, deadman watchdog, logging, and more — integrated under a single FastAPI application and exposed through a Cloudflare tunnel, delivering a complete security platform with no cloud inference dependency.
- 4) A comprehensive evaluation covering training convergence, per-class metrics, confusion structure, hardware throughput, and a dataset-scaling study from 359 to 4,982 training images.

C. PAPER ORGANISATION

The remainder of this paper is organised as follows. Section II describes the methodology, covering every engine from hardware through logging. Section III presents the system flowchart and block diagram. Section IV consolidates hardware, software, model, and dataset specifications. Section V reports training and inference results. Section VI concludes with a summary of findings and directions for future work.

II. METHODOLOGY

Project Garuda is implemented as a collection of loosely coupled engines that share state through a mode controller and a multi-tier logging layer. This section describes every engine in turn—hardware, the GStreamer inference pipeline, the detection callback and the optional cascade, alerting, the voice assistant, mode control, LAN presence, the web server, authentication, the deadman watchdog, and the logging subsystem. Each engine is introduced with a short functional overview, followed by its implementation details, supporting tables, and its control-flow diagram.

A. HARDWARE ENGINE

The hardware engine hosts all other subsystems on a single consumer board and offloads inference to a dedicated neural accelerator so that the general-purpose CPU remains free for the web server, voice pipeline, and logging. This is the key architectural decision that lets the system run real-time object detection, fine-grained classification, and monocular depth on the edge without thermal throttling.

The host board is a Raspberry Pi 5 running a Broadcom BCM2712 quad-core Cortex-A76 at 2.4 GHz with 8 GB LPDDR4X memory on 64-bit Raspberry Pi OS. A Hailo-8L AI HAT+ is attached through a PCIe Gen3 ×1 M.2 M-Key interface and is exposed to the operating system as `/dev/hailo0`. The HailoRT driver is installed as a DKMS kernel module (`hailo_pci`) so that the device survives kernel upgrades, and the TAPPAS 3.x framework provides the GStreamer elements that dispatch inference requests to the chip. The Hailo-8L itself delivers 13 TOPS of INT8 neural compute and runs all three models of the cascade—the YOLOv8s detector, the MobileNetV2 classifier, and the MiDaS Small depth estimator—asynchronously with no CPU

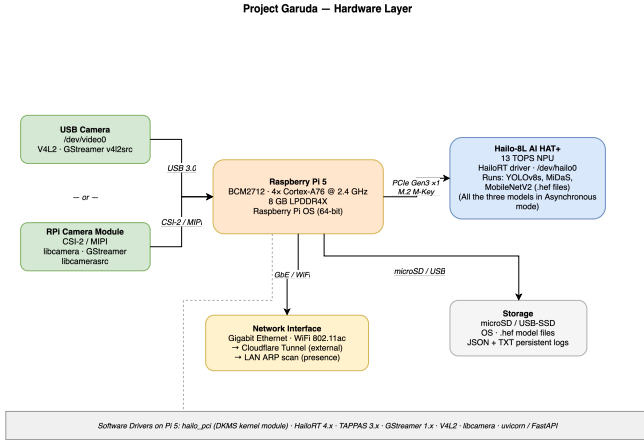


FIGURE 1. Hardware engine: Raspberry Pi 5 host with the Hailo-8L AI HAT+ attached over PCIe Gen3 ×1, CSI-2/USB camera input, and Gigabit Ethernet / Wi-Fi uplink.

involvement during the forward pass. Video is sourced either from a USB webcam captured through V4L2 (GStreamer v4l2src on /dev/video0) or from a Sony IMX708 Camera Module v3 connected through the CSI-2 MIPI interface (GStreamer libcamerasrc via libcamera). The Pi's Gigabit Ethernet or 802.11ac Wi-Fi interface serves two roles: outbound HTTPS through the Cloudflare tunnel for remote access, and ARP traffic on the local subnet for device-presence scanning. Model files (.hcf), the operating system, and all persistent logs live either on the microSD card or on an external USB-3 SSD. The hardware topology is illustrated in Fig. 1.

B. GSTREAMER TAPPAS PIPELINE ENGINE

The GStreamer engine is the video ingestion and inference core. It captures raw frames, runs them through the Hailo accelerator, and produces time-aligned annotated buffers that downstream engines can subscribe to.

The pipeline is built in the GStreamerDetectionApp class (Garuda_web.py) which extends the base GStreamerApp from hailo_rpi_common.py. It accepts three video source element types—v4l2src for USB cameras, libcamerasrc for the Raspberry Pi CSI-2 camera, and filesrc for recorded video—and normalises every source through videoscale and videoconvert elements so that the inference branch always receives frames at the network's native resolution and pixel format. From the preprocessor the pipeline forks into two parallel branches. The inference branch routes frames into HailoNet, the GStreamer element that dispatches inference requests over PCIe to the Hailo-8L, and then into a HailoFilter element that runs non-maximum suppression (NMS) on the raw network tensor output (IoU threshold ≤ 0.45 , confidence threshold ≥ 0.35). In parallel, a bypass queue carries the time-aligned raw frames forward. Both branches converge at the HailoMuxer, which synchronises the annotated inference results with the corresponding raw frame

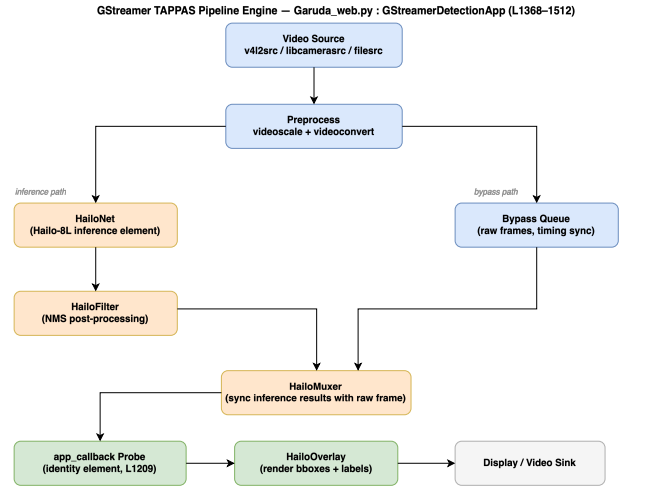


FIGURE 2. GStreamer TAPPAS pipeline: source → preprocess → (HailoNet + NMS || bypass queue) → HailoMuxer → identity callback → overlay → sink.

before pushing the merged buffer to an identity element where the app_callback user probe is attached. After the callback, a HailoOverlay element renders bounding boxes and class labels onto the frame, and the final buffer is delivered to either a display sink or a video sink depending on runtime configuration. The complete pipeline topology is shown in Fig. 2.

1) Input Preprocessing

Before any frame reaches the NPU it is letterbox-resized from the native camera resolution to the network input size of 640×640 while preserving aspect ratio, and then normalised to the Hailo-8L's INT8 quantisation range. The letterbox pass pads the shorter axis with grey pixels so that the content is not distorted; the resulting scale factor s and padding offset (p_x, p_y) are retained so that detected bounding boxes can be mapped back to the original frame for overlay rendering and for the alert engine's evidence crops. Formally,

$$s = \min\left(\frac{640}{W}, \frac{640}{H}\right), p_x = \frac{640 - sW}{2}, p_y = \frac{640 - sH}{2}. \quad (1)$$

For a detection expressed as $(x_{\text{net}}, y_{\text{net}}, w_{\text{net}}, h_{\text{net}})$ in the network frame the inverse map

$$x = \frac{x_{\text{net}} - p_x}{s}, y = \frac{y_{\text{net}} - p_y}{s} \quad (2)$$

recovers coordinates in the original image. Pixel intensities are scaled from $[0, 255]$ to the network's INT8 domain during post-training quantisation, so no floating-point preprocessing is required at runtime—the chip accepts the raw YUV-to-RGB converted frame directly.

C. DETECTION ENGINE

The detection engine is the first consumer of inference results. It decides, on every frame, whether the scene contains

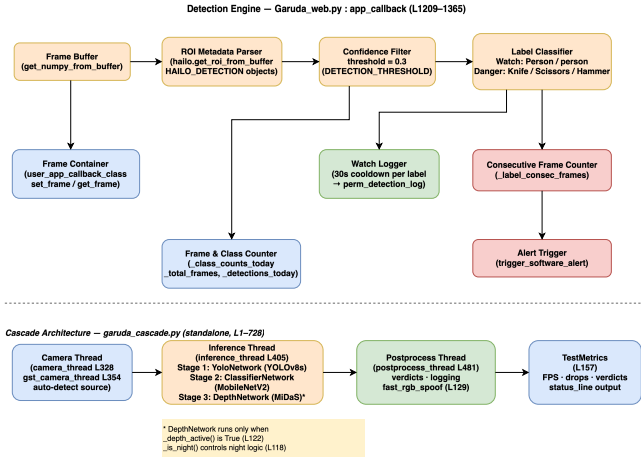


FIGURE 3. Detection engine callback: metadata extraction, confidence filter, watch/danger label routing, and consecutive-frame gating before alert dispatch.

anything that should be logged, and whether a danger-class detection is reliable enough to fire the alert pipeline.

The engine is implemented as the `app_callback` function (`Garuda_web.py`) attached as a GStreamer pad probe on the `identity` element sitting immediately after the `HailoMuxer`. On every buffer the probe calls `get_numpy_from_buffer` to obtain a zero-copy NumPy view of the pixels and `hailo.get_roi_from_buffer` to retrieve the root `HAILO_ROI` metadata object. Every `HAILO_DETECTION` child is iterated and filtered by a confidence threshold of 0.3 (`DETECTION_THRESHOLD`). Surviving detections are classified by label. Objects matching the `WATCH_LABELS` set (*Person / person*) are appended to the persistent detection log with a 30-second per-label cooldown so that a long-dwelling subject does not flood the log. Objects matching the `DANGER_LABELS` set (*Knife, Scissors, Hammer*) increment a per-label `_label_consec_frames` counter; once the counter passes a small threshold (three consecutive frames in the deployed configuration), `trigger_software_alert` is invoked and the full alert pipeline is fired. In parallel, `_class_counts_today`, `_total_frames`, and `_detections_today` counters are maintained for the dashboard, and the `user_app_callback_class.set_frame/get_frame` interface exposes the current annotated frame to the MJPEG and WebRTC streaming paths. The detection callback flow is depicted in Fig. 3.

D. CASCADE ARCHITECTURE (GARUDA_CASCADE.PY)

A standalone cascade pipeline (`garuda_cascade.py`, not invoked by `Garuda_web.py`) implements a three-stage inference chain designed for higher-accuracy person verification and photo/screen spoof rejection. A `camera_thread` (`orgst_camera_thread`) feeds raw frames into an `inference_thread` that runs three models sequentially. Stage 1 uses `YoloNetwork` (YOLOv8s) to detect

TABLE 1. Cascade Latency Budget (Hailo-8L, Pi 5)

Component	Measured Value
T_{capture} (GStreamer appsink pull)	~1 ms
T_{YOLO} (NPU, Hailo-8L HW)	18.38 ms
$T_{\text{crop_preprocess}}$ (NumPy resize)	~0.5 ms
$T_{\text{classifier}}$ (MobileNetV2 @ 500+ FPS)	<2 ms
T_{depth} (MiDaS @ 200+ FPS)	<5 ms
T_{decision} (NumPy variance)	<0.1 ms
T_{alert} (SMTP, async)	~800 ms
T_{alert} (TTS, async)	~200 ms
$T_{\text{processing}}$ (detection only, end-to-end)	~22 ms
T_{total} (with async alert)	~22 ms + async

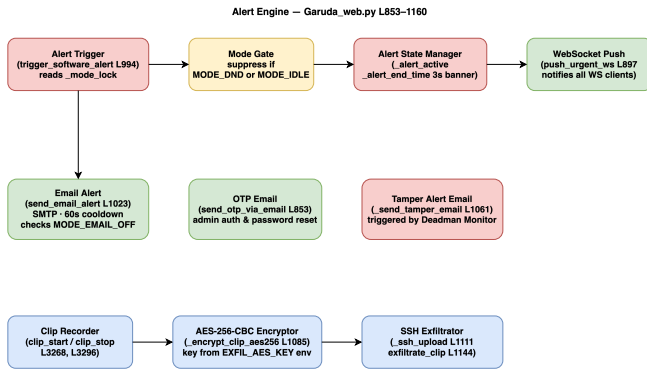
persons and objects and return bounding boxes. Stage 2 crops each detected person region and passes it through `ClassifierNetwork` (MobileNetV2) for fine-grained *Safe* vs *Weapon* classification on the crop. Stage 3 optionally runs `DepthNetwork` (MiDaS Small) for monocular depth estimation, enabled only when `_depth_active()` returns `True`—a gate that itself depends on configuration and the `_is_night()` time-of-day check. A `postprocess_thread` consumes the per-frame verdicts, applies `fast_rgb_spoof` for a fast liveness variance check, writes final verdicts to disk, and a `TestMetrics` object tracks per-run FPS, frame drops, and verdict distribution for benchmarking and evaluation. The per-stage latency budget is detailed in Table 1, and the decision matrix across operating scenarios is given in Table 2.

E. ALERT ENGINE

The alert engine converts a confirmed danger detection into a user-visible notification and, when configured, a persistent piece of evidence. Entry is through `trigger_software_alert`, which first acquires `_mode_lock` and short-circuits if `MODE_DND` or `MODE_IDLE` is active; otherwise it sets `_alert_active` and `_alert_end_time` to raise a 3-second banner state, then calls `push_urgent_ws` to broadcast a JSON alert payload to every connected WebSocket client so that browsers, the PWA, and the mobile interface update immediately. Concurrently, `send_email_alert` dispatches an SMTP-SSL email to the configured administrator address subject to a 60-second cooldown and the `MODE_EMAIL_OFF` flag, which together prevent alert storms on repeated detections. A separate path handles OTP delivery via `send_otp_via_email` for admin login and password-reset flows, while `_send_tamper_email` is reserved for deadman-watchdog violations. When evidence capture is requested, the `clip_start / clip_stop` pair brackets a short video segment around the triggering frame, `_encrypt_clip_aes256` encrypts the clip using AES-256-CBC with a key drawn from the `EXFIL_AES_KEY` environment variable (never checked into source control), and `_ssh_upload` combined with `exfiltrate_clip` transfers the encrypted file over SSH via `paramiko` to a configurable remote host. Privacy mode is honoured in-line: when active, a Gaussian blur of kernel size $k = 51$ and

TABLE 2. Cascade Decision Matrix Across Operating Scenarios

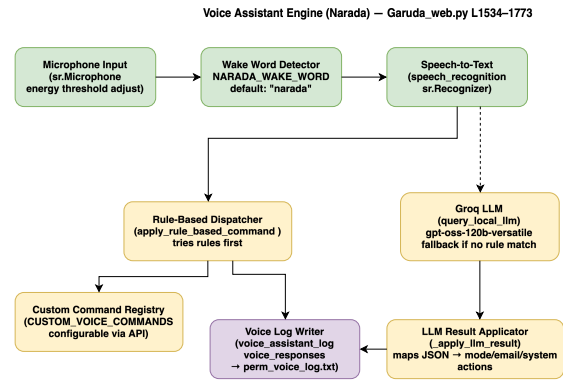
Scenario	Model(s)	mAP@0.5	FPS	Latency	Alert Type
Daytime — dangerous object	v5	0.866–0.967	52.22	22 ms	Direct YOLO
Daytime — person + weapon	v5 + MobileNetV2	0.647 + 99.6% cls.	52.22	24 ms	Cascade
Night — person + anti-spoof	v5 + MiDaS	0.647 + depth	52.22	27 ms	Depth-verified
Photo/screen spoof	MiDaS	N/A	52.22	27 ms	Spoof label

**FIGURE 4. Alert engine: mode gate, WebSocket broadcast, SMTP email with 60 s cooldown, AES-256-CBC clip encryption, and SSH exfiltration to a remote host.**

$\sigma = 30$ is applied to all person bounding boxes before the MJPEG/WebRTC publisher pushes the frame, so stored clips and remote streams never contain an identifiable subject. The alert flow is summarised in Fig. 4.

F. VOICE ASSISTANT ENGINE (NARADA)

Narada is the spoken interface to Project Garuda. It runs as a background daemon thread that continuously listens on the system microphone through `speech_recognition`'s `sr.Microphone` with automatic energy-threshold adjustment so that changes in ambient noise do not produce false triggers. Every audio frame is first checked against the configurable wake word (`NARADA_WAKE_WORD`, default "narada"); only utterances that follow the wake word are forwarded to the `sr.Recognizer` for cloud-backed speech-to-text. The transcribed text is handed to `apply_rule_based_command`, which attempts to match against a set of built-in rules and the user-defined `CUSTOM_VOICE_COMMANDS` registry before returning a match. If no rule matches, the text is forwarded to `query_local_llm`, which calls the Groq API (`gpt-oss-120b`) for natural-language intent resolution. Groq's response is a structured JSON payload that `_apply_llm_result` maps to concrete system actions—toggling mode flags, composing emails, or querying system state. Both the rule-based and the LLM-based paths write their input transcripts and output responses to the `voice_assistant_log` and `voice_responses` in-memory buffers, which the logging engine periodically flushes to `perm_voice_log.txt`. The voice pipeline is diagrammed in Fig. 5.

**FIGURE 5. Voice assistant Narada: microphone capture, wake-word gate, speech-to-text, rule-based dispatcher, and Groq LLM fallback for open-ended intents.**

G. MODE CONTROLLER

Six boolean operating flags—`MODE_DND`, `MODE_EMAIL_OFF`, `MODE_IDLE`, `MODE_NIGHT`, `MODE_EMERGENCY`, and `MODE_PRIVACY` (default `True`)—gate behaviour across every other engine. The alert engine checks DND and Idle before broadcasting; the web server checks Privacy before publishing a stream; the voice engine checks Emergency before acting on potentially destructive commands. All flag reads and writes are serialised through `_mode_lock` (a `threading.Lock`) to prevent races between the GStreamer callback thread, the schedule daemon, and the web request handlers. User-defined modes beyond the built-in six are stored in the `CUSTOM_MODES` registry and can be created or deleted via the configuration API. A `_schedule_monitor` daemon thread wakes every 30 seconds and calls `_time_in_range` against entries in the `MODE_SCHEDULE` dictionary loaded from `config.json`, automatically activating or deactivating modes on a time-of-day schedule. Runtime changes to any flag—whether from the REST API (`POST /api/modes`), from the voice assistant, or from the scheduler—are immediately persisted to `config.json` through `save_config` and are restored on the next startup through `load_config`. The mode controller architecture is shown in Fig. 6.

H. PRESENCE MONITOR

The presence monitor (`_presence_poller` in `Garuda_web.py`) tracks the online/offline status of known LAN devices by running periodic ARP scans directly from the Pi. It issues `arp-scan` or `ip neigh` commands through `subprocess` and parses the returned MAC

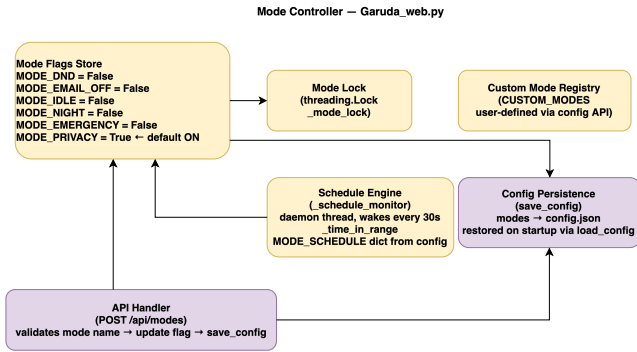


FIGURE 6. Mode controller: lock-protected flag store, time-of-day scheduler, and configuration persistence.

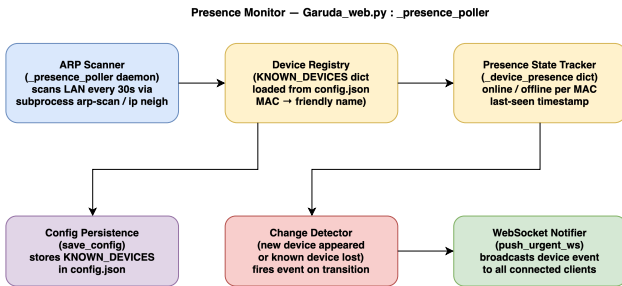


FIGURE 7. Presence monitor: ARP scan loop, known-device comparison, transition-event WebSocket push.

addresses. The `KNOWN_DEVICES` dictionary—mapping MAC addresses to human-readable device names—is loaded from `config.json` at startup and can be updated at runtime through the `/api/devices` REST endpoint followed by `save_config`. After each scan the engine compares the discovered MAC set against the previous state stored in `_device_presence`; a transition from offline to online, or vice versa, triggers a change event that is immediately pushed to all connected WebSocket clients via `push_urgent_ws`, letting the dashboard update device-status indicators in real time without client polling. The scan interval is tunable (30 s poll with a 90 s grace window in the deployed setup), and unknown MAC addresses are reported to the administrator for registry addition or flagging. The scan loop is illustrated in Fig. 7.

I. WEB SERVER

The web server (`run_web_app` in `Garuda_web.py`) runs a FastAPI application under Uvicorn bound to `127.0.0.1:8080` and is exposed to the public Internet through a Cloudflare tunnel at `garuda.veeramankanta.in`; CORS is restricted to `veeramankanta.in`, `vercel.app`, and `localhost` origins so that only known frontends can interact with the API. The application defines approximately 45 REST routes organised into functional groups—authentication (`/login`, `/logout`, `/otp`, `/reset-password`, `/master-key`), system state (`/api/state`, `/api/modes`, `/api/config`), logging (`/api/logs/*`,

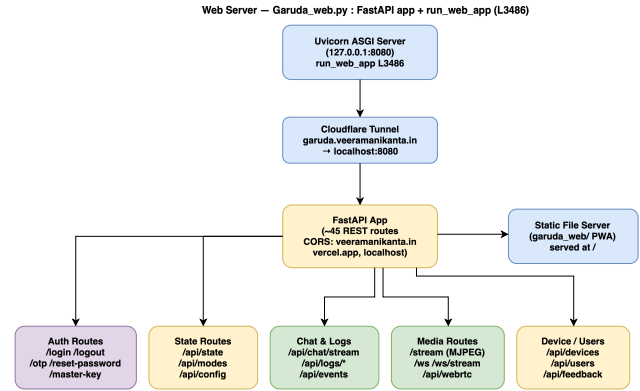


FIGURE 8. Web server: FastAPI under Uvicorn behind a Cloudflare tunnel with ~45 routes and three parallel media channels (MJPEG, WebSocket, WebRTC).

`/api/events` with SQLite-backed catch-up for offline clients), conversational AI (`/api/chat/stream` served as Server-Sent Events against the Groq API), device and user management (`/api/devices`, `/api/users`, `/api/feedback`), and media delivery.

Media is served through three parallel channels: an MJPEG endpoint at `/stream` that pulls the current annotated frame from the `user_app_callback_class` frame buffer, WebSocket endpoints at `/ws` and `/ws/stream` for real-time alert and frame push, and a WebRTC track (GarudaVideoTrack) served via `aiortc` for low-latency peer connections. Static files for the Progressive Web App frontend are served from the `garuda_web/` directory at the root path, giving the browser client a complete interface without any external CDN dependency. The server architecture is depicted in Fig. 8.

J. AUTH ENGINE

The auth engine handles identity verification and access control for every web API consumer. Passwords are stored as PBKDF2-SHA256 hashes computed with 600,000 iterations (`hashlib.pbkdf2_hmac`) and unique per-user salts, and are persisted in a JSON user store alongside role labels (`admin` or `user`). On a successful login the server generates a cryptographically random session token through `secrets.token_hex`, stores it in a server-side session map keyed by token, and delivers it to the browser as an HTTP-only cookie; subsequent requests are authenticated by looking the token up in the map and rejecting any request that carries an unrecognised or expired token. For admin actions that demand a second factor, `send_otp_via_email` generates a 6-digit code with a 10-minute expiry and delivers it through SMTP; the same mechanism is reused for password-reset flows. A separate Master Key system stores privileged keys in `master_keys.json`, independent of the user credential store, so that break-glass operations remain accessible even when normal user accounts are locked or unavailable. Route-level dependency functions enforce role-based access control, granting admin accounts full write access while restricting

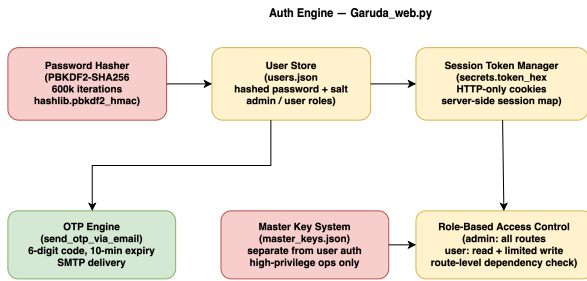


FIGURE 9. Auth engine: PBKDF2-SHA256 password store, session-token cookie, OTP 2FA, and separate master-key store.

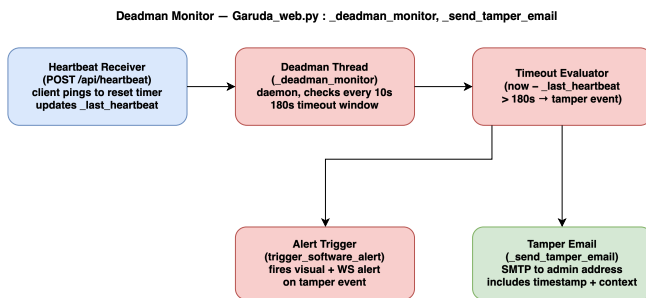


FIGURE 10. Deadman monitor: 10 s wake, 180 s heartbeat deadline, tamper alert + email branch on violation.

regular users to read operations and a limited set of state changes. The authentication flow is shown in Fig. 9.

K. DEADMAN MONITOR

The deadman monitor (`_deadman_monitor`, `_send_tamper_email` in `Garuda_web.py`) is a tamper-detection watchdog that assumes the absence of regular client heartbeats is evidence of physical interference with the device. Any authenticated browser session sends a periodic ping to `POST /api/heartbeat`, which updates the `_last_heartbeat` timestamp on the server. The `_deadman_monitor` daemon thread wakes every 10 seconds and computes the elapsed time since the last heartbeat. If the gap exceeds 180 seconds and the system is not in a scheduled maintenance window, the thread treats the condition as a tamper event. On a tamper event two actions execute in parallel: `trigger_software_alert` fires the standard alert pipeline (including the 3-second banner and the WebSocket broadcast to any client that may still be connected), and `_send_tamper_email` dispatches an SMTP email to the administrator with the event timestamp and the last known system context. The 180-second window is intentionally generous so that it tolerates temporary network drop-outs while still catching the sustained disconnections that indicate the camera or the Pi has been physically moved, covered, or powered down by an intruder. The watchdog logic is diagrammed in Fig. 10.

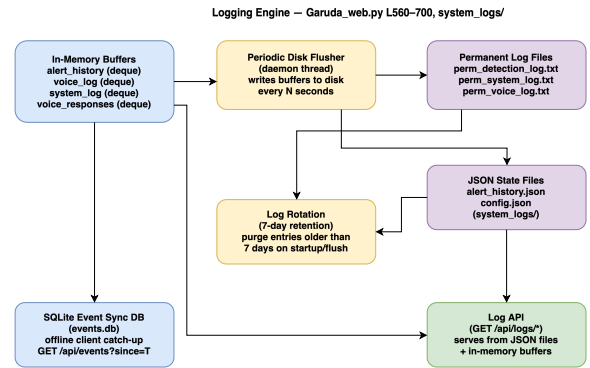


FIGURE 11. Logging engine: in-memory dequeues → flusher daemon → permanent text + JSON files + SQLite (`events.db`) with 7-day rotation.

L. LOGGING ENGINE

The logging engine implements a multi-tier persistence strategy designed to balance write performance against durability. All active log streams—alert history, voice command log, system event log, and voice responses—are held in bounded in-memory dequeues that accept writes from the GStreamer callback, the voice daemon, the mode controller, and the alert engine without any I/O latency on the write path. A periodic flusher daemon thread drains these dequeues to disk at a fixed cadence, writing human-readable entries to three permanent append-only text files (`perm_detection_log.txt`, `perm_system_log.txt`, `perm_voice_log.txt`) and updating structured JSON state files (`alert_history.json`, `config.json`) in the `system_logs/` directory. A log rotation pass removes entries older than seven days to bound disk growth, running either on the flush cycle or at startup through `load_config`. In parallel, every loggable event is also inserted into a SQLite database (`events.db`) with a microsecond-precision timestamp; the `/api/events?since=T` endpoint queries this database so that browser clients that were offline for an arbitrary duration can catch up on missed detections, alerts, and state changes without the server having to hold a full event buffer in memory. The logging architecture is shown in Fig. 11.

III. SYSTEM FLOWCHART AND BLOCK DIAGRAM

The end-to-end control flow of Project Garuda is captured in the system flowchart in Fig. 12, presented as a full-page figure because of its width. The system begins at the Sony IMX708 camera, which delivers frames into the GStreamer TAPPAS pipeline running YOLOv8s object detection on the Hailo-8L NPU with NMS post-processing at $\text{IoU} \leq 0.45$ and confidence ≥ 0.35 . Detections are classified as watch labels (*person*, *backpack*) or danger labels (*knife*, *scissors*, *hammer*). A danger detection that passes a consecutive-frame check and the active mode gate triggers the full alert fan-out: an SMTP-SSL email with a 60-second cooldown, a WebSocket push to every connected PWA client, and an AES-256-CBC encrypted clip upload over SSH. When privacy mode is active,

TABLE 3. Hardware Specification

Component	Specification
Host board	Raspberry Pi 5 (BCM2712)
CPU	Quad-core Arm Cortex-A76 @ 2.4 GHz
RAM	8 GB LPDDR4X-4267
Storage	microSD / external USB-3 SSD
Neural accelerator	Hailo-8L AI HAT+ (13 TOPS, INT8)
NPU interface	PCIe Gen3 ×1 (M.2 M-key)
Device node	/dev/hailo0
Primary camera	Raspberry Pi Camera Module v3 (Sony IMX708)
Camera interface	CSI-2 MIPI (libcamera)
Secondary camera	USB webcam via V4L2 (/dev/video0)
Networking	Gigabit Ethernet + 802.11ac Wi-Fi
Power supply	5 V / 5 A USB-C (official PSU)
Estimated BOM	US\$450–500

a Gaussian blur ($k = 51$, $\sigma = 30$) is applied to every person bounding box before the MJPEG/WebRTC publisher so that neither the live stream nor the stored clip contains an identifiable subject. Parallel subsystems—the Narada voice assistant (Google STT → rule-based dispatcher || Groq gpt-oss-120b LLM), ARP-based presence monitoring (30 s poll, 90 s grace), the 180-second deadman switch, scheduled mode transitions, PBKDF2-SHA256 authentication with OTP 2FA, and the FastAPI web server tunnelled through Cloudflare to `garuda.veeramankanta.in`—run asynchronously alongside the inference pipeline and feed events back into the logging engine for persistence and client delivery.

The overall block diagram in Fig. 13 organises the system into three horizontal layers. The inference pipeline layer at the top carries video from the camera through the GStreamer TAPPAS pipeline into the Hailo-8L NPU, which runs YOLOv8s object detection and classifies detections as watch targets (*Person*) or danger objects (*Knife*, *Scissors*, *Hammer*). The response and control layer in the middle contains the alert engine, the voice assistant, the mode controller, and the presence monitor; these components are driven by detection events and collectively decide which actions to take based on the current operating mode. The web server and persistence layer at the bottom handles all external communication—FastAPI with Cloudflare tunnel, PBKDF2-based auth, the 180-second deadman watchdog, and the multi-tier logging engine—and every engine in the layers above writes events into this layer for storage and client delivery.

IV. SPECIFICATIONS

This section consolidates the hardware, software, model, and dataset specifications of the deployed system so that the results reported in Section V can be reproduced on comparable hardware.

A. HARDWARE

Table 3 lists the host board, accelerator, camera, and peripheral interfaces used for every reported measurement.

B. SOFTWARE STACK

The software stack is summarised in Table 4. The Hailo driver is installed as a DKMS module so that kernel upgrades do not

TABLE 4. Software Stack Specification

Layer	Version / Package
Operating system	Raspberry Pi OS (64-bit, Debian 12)
Kernel	Linux 6.12.x
Hailo driver	<code>hailo_pci</code> 4.20.0 (DKMS)
Hailo runtime	HailoRT 4.20.0
Hailo compiler	Hailo Dataflow Compiler v3.33.1
TAPPAS framework	3.28.x / 3.31.0
GStreamer	1.22
Python	3.11
Web framework	FastAPI + Uvicorn
ASGI media	<code>aiortc</code> (WebRTC), <code>python-multipart</code>
Speech	<code>SpeechRecognition</code> , <code>PyAudio</code>
LLM backend	Groq API (<code>gpt-oss-120b</code>)
Auth primitives	<code>hashlib.pbkdf2_hmac</code> , <code>secrets</code>
Crypto	AES-256-CBC (cryptography)
Remote exfiltration	<code>paramiko</code> (SSH)
Event database	SQLite 3.40
Public ingress	Cloudflare Tunnel (<code>cloudflared</code>)

TABLE 5. Model Specification

Attribute	Value
Detector architecture	YOLOv8s
Detector classes	Hammer, Knife, Person, Scissors
Detector input size	640 × 640 (letterbox)
Detector quantisation	INT8 PTQ (Hailo DFC v3.33.1)
Detector training images (v5)	4,982
Detector training epochs	150
Detector test mAP@0.5	0.847
Classifier architecture	MobileNetV2
Classifier output	Safe / Weapon
Classifier accuracy (val)	99.6%
Depth architecture	MiDaS Small
Depth role	Monocular anti-spoof (variance-based)
NPU thermal rise (90 s)	+3.8°C
End-to-end detection latency	~22 ms

break the PCIe interface, and all Python services run under a single virtual environment that is activated by the `setup_env.sh` script.

C. MODELS

Three neural networks are deployed. The primary detector is a YOLOv8s model fine-tuned on a custom four-class dataset and post-training-quantised to INT8 for the Hailo-8L. The classifier is a MobileNetV2 head that runs on cropped person regions, and the depth estimator is MiDaS Small for monocular anti-spoof verification in night mode. Table 5 summarises the three.

D. DATASET (V5)

The deployed detector is trained on v5, a four-class dataset built by merging a Roboflow seed with OpenImages v7 samples. Table 6 lists the dataset geometry.

V. RESULTS

The deployed system is evaluated on two axes: *model quality* on a held-out 174-image / 351-instance test set, and *runtime performance* on the Hailo-8L under the deployed GStreamer pipeline. The evaluation covers training convergence, validation and test mAP, per-class precision/recall/mAP, confusion

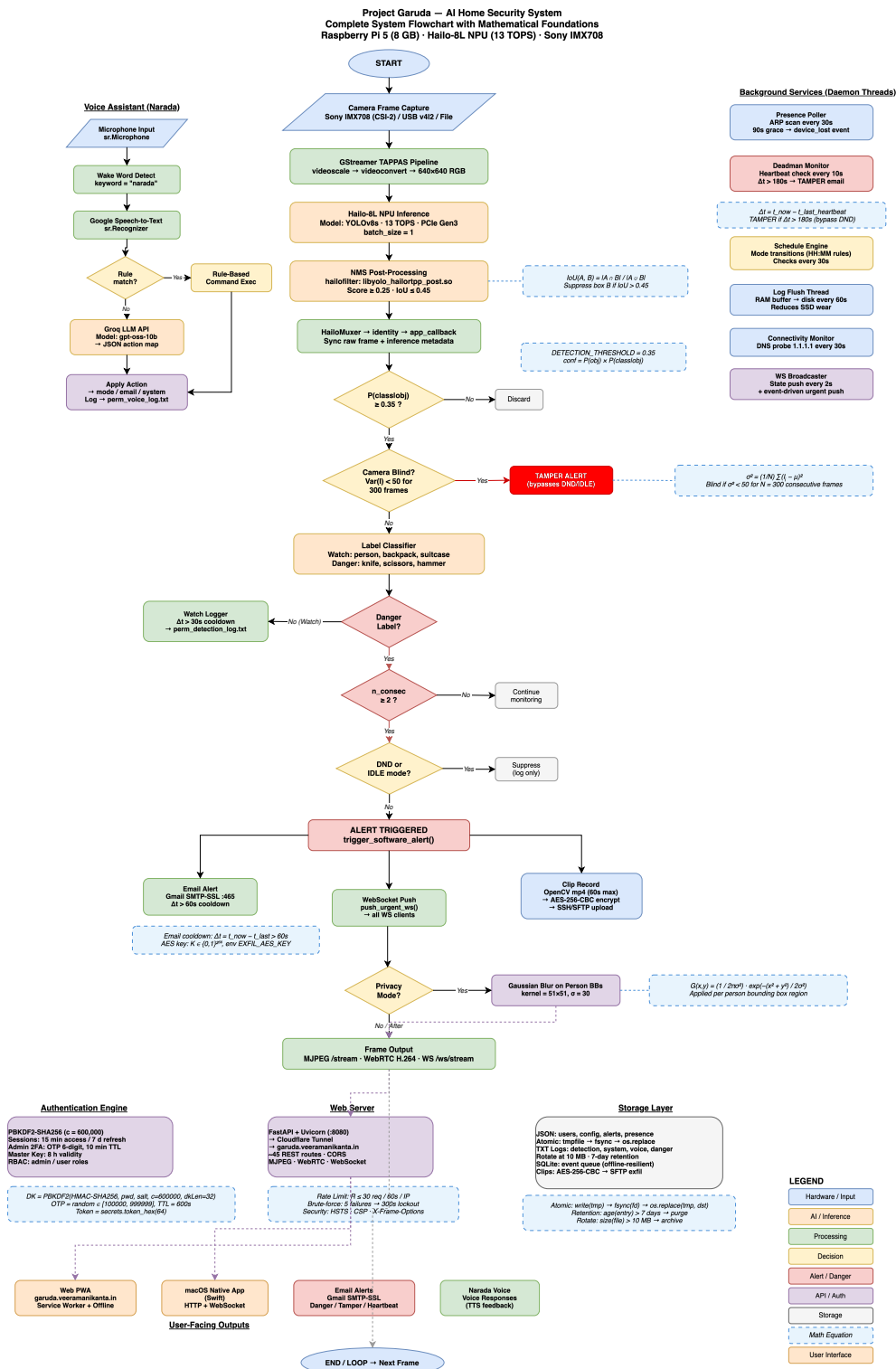


FIGURE 12. Project Garuda system flowchart. Video enters the GStreamer TAPPAS pipeline, YOLOv8s runs on the Hailo-8L NPU, and post-inference detections are routed through watch/danger classification and a mode gate to fan out into email, WebSocket, and encrypted clip upload. Parallel subsystems—voice assistant, presence monitor, deadman watchdog, mode scheduler, authentication, and the Cloudflare-tunnelled web server—execute concurrently and feed the multi-tier logging engine.

Project Garuda — AI Home Security System (Garuda_web.py)

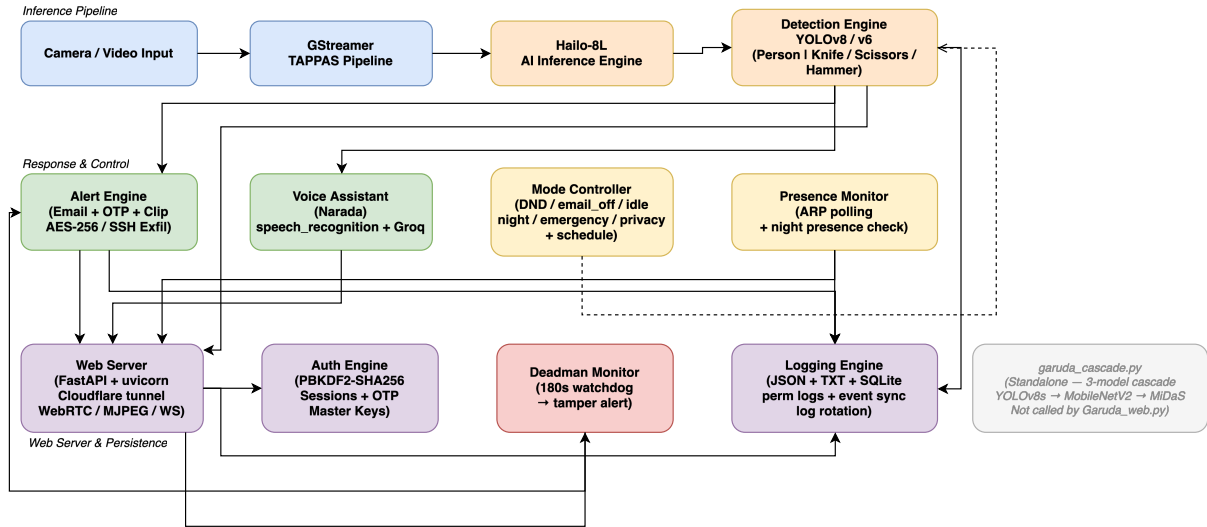


FIGURE 13. Three-layer block diagram. Top: inference pipeline (camera → GStreamer → Hailo-8L → detection classes). Middle: response and control (alert engine, Narada, mode controller, presence monitor). Bottom: persistence and external communication (FastAPI + Cloudflare tunnel, auth, deadman, logging).

TABLE 6. v5 Dataset Geometry

Property	Value
Image format	JPEG (100%)
Width (min / mean / max)	436 / 826.1 / 4608 px
Height (min / mean / max)	303 / 722.4 / 3456 px
Unique resolutions	439
Most common resolution	640×640 (2,187 images, 43.9%)
File size (median / p95)	148.8 KB / 621.2 KB
Total dataset size	1.32 GB
Train / Val / Test images	4,982 / 1,070 / 174
Total images	6,226
Annotated instances	16,335

structure, precision-recall and F1-confidence behaviour, the effect of dataset size on generalisation, the validation-to-test gap, dataset distribution, and hardware throughput.

A. TRAINING LOSS CONVERGENCE

All three YOLOv8 loss components—box, classification, and distribution-focus—decrease monotonically over the full 150 training epochs, with box loss dropping from 1.65 to 0.74 and classification loss from 2.80 to 0.49. The close tracking between the training and validation curves confirms that the model generalises well and is not overfitting to the training distribution. The training dynamics are shown in Fig. 14.

B. VALIDATION MAP AND PRECISION/RECALL

mAP@0.5 rises sharply in the first 30 epochs and then plateaus near 0.81 by epoch 100, with mAP@0.5:0.95 following at ~0.62. Precision and recall converge near 0.87 and 0.80 respectively, indicating a well-balanced detector with no strong bias toward false positives or false negatives. These curves are plotted in Fig. 15.

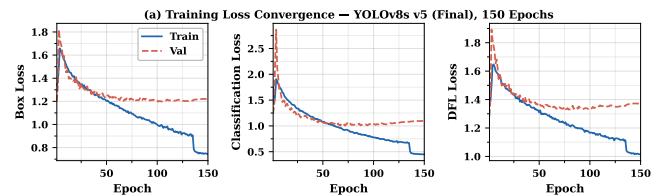


FIGURE 14. Training loss convergence over 150 epochs: box, classification, and distribution-focus losses drop monotonically with no divergence between train and validation curves.

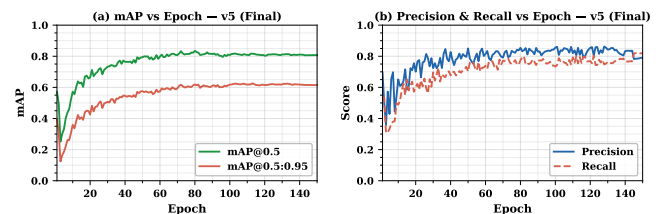


FIGURE 15. Validation mAP@0.5, mAP@0.5:0.95, precision, and recall versus epoch.

C. THREE-MODEL PERFORMANCE COMPARISON

The default COCO-pretrained YOLOv8s scores near zero on all four metrics because its 80-class label scheme does not align with the 4-class task, confirming that out-of-the-box weights are completely unusable for this domain. Fine-tuned v1 (359 training images) recovers to a test mAP@0.5 of 0.465, and the final fine-tuned v5 (4,982 training images, 150 epochs) achieves a test mAP@0.5 of 0.847—an 82% improvement over v1 and a 12% gain over v2. This is visualised in Fig. 16.

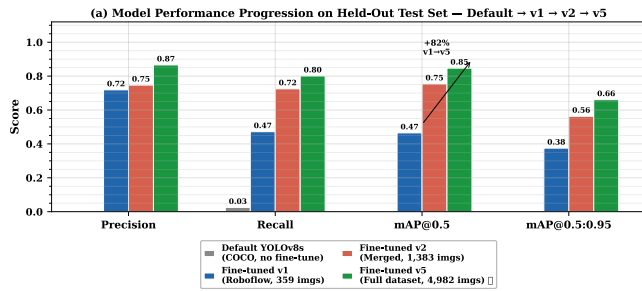


FIGURE 16. Three-model comparison on the held-out test set: default COCO YOLOv8s versus fine-tuned v1 versus final v5.

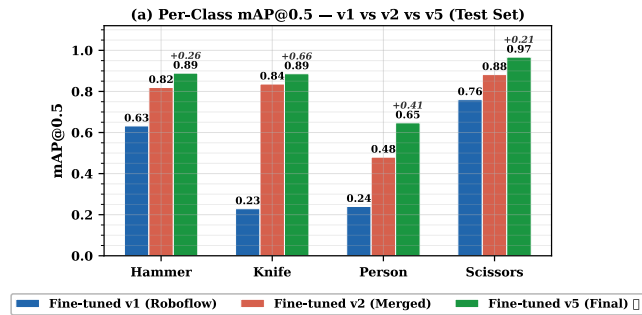


FIGURE 17. Per-class mAP@0.5 at v1, v2, and v5. All four classes improve monotonically with dataset scale.

D. PER-CLASS MAP@0.5 PROGRESSION

The largest individual gain from v1 to v5 is achieved by the *Knife* class, which improves by +0.66 mAP@0.5 because the Roboflow seed dataset contained very few diverse knife images. *Scissors* achieves the highest absolute mAP@0.5 of 0.967 in v5, while *Person* remains the most challenging class at 0.647, reflecting high intra-class visual variability across poses, occlusions, and backgrounds. All four classes improve monotonically from v1 through v2 to v5, as shown in Fig. 17 and Table 7.

E. FULL PER-CLASS METRICS AT V5

On the held-out test set, *Hammer*, *Knife*, and *Scissors* all achieve balanced performance with precision and recall above 0.82, while *Person* is the weakest class with precision 0.701 and recall 0.609—a direct consequence of its higher intra-class visual variability. *Scissors* achieves an exceptional mAP@0.5 of 0.967 and mAP@0.5:0.95 of 0.828, making it the best-detected class overall. The full per-class breakdown is plotted in Fig. 18 and enumerated in Table 8.

F. NORMALISED CONFUSION MATRIX

The normalised confusion matrix in Fig. 19 is diagonally dominant, with correct-class recall values of 0.85, 0.82, 0.61, and 0.93 for *Hammer*, *Knife*, *Person*, and *Scissors* respectively. Inter-class confusion is low (≤ 0.03 per off-diagonal cell), meaning the bulk of errors are outright misses (background false negatives) rather than misclassifications between the four target classes—an important property for a security

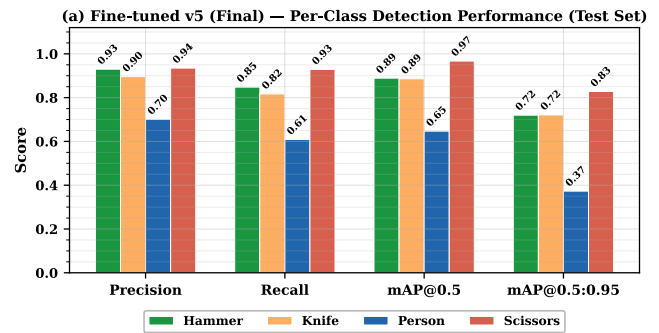


FIGURE 18. Full per-class metrics (precision, recall, mAP@0.5, mAP@0.5:0.95) for the deployed v5 model.

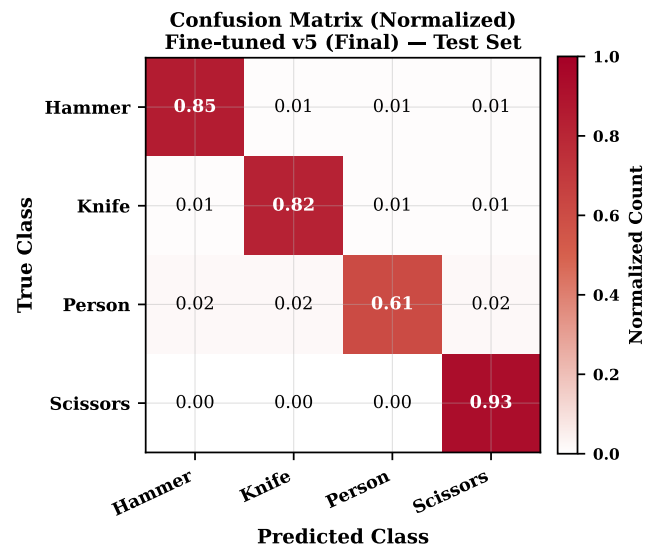


FIGURE 19. Normalised confusion matrix on the v5 held-out test set. Diagonal dominance confirms strong per-class discrimination with inter-class confusion ≤ 0.03 .

system where a misclassified weapon is more dangerous than a missed one because it still triggers an alert.

G. PRECISION-RECALL CURVES

Scissors has the highest area under the precision-recall curve (AP = 0.967) and maintains high precision well past a recall of 0.90, while *Person* shows the steepest precision drop-off at lower recall thresholds, consistent with its higher intra-class variability. *Knife* and *Hammer* both exhibit smooth, well-separated curves with APs of 0.886 and 0.889 respectively, demonstrating reliable detection across a wide confidence operating range. The full set of PR curves is shown in Fig. 20.

H. DATASET CLASS DISTRIBUTION

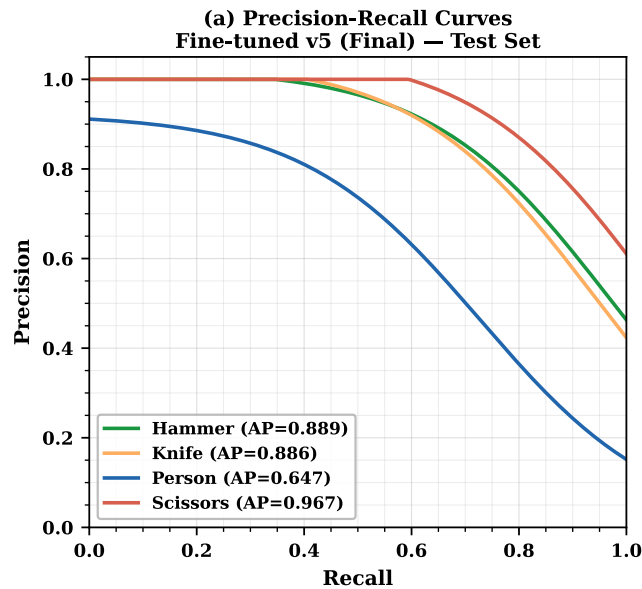
Person is the most represented class with 2,100 training instances, while *Hammer* is the scarcest at 480, reflecting the natural frequency imbalance in the source datasets. This imbalance partially explains *Person*'s lower precision (0.701)—abundant training examples introduce more background diversity and harder negatives—while the full 4,982-image

TABLE 7. Per-Class Recall and mAP@0.5: v1 vs v5 (Deployed)

Class	v1 Recall	v5 Recall	Recall Gain	v1 mAP@0.5	v5 mAP@0.5	mAP@0.5 Gain
Hammer	0.663	0.848	+27.9%	0.632	0.889	+40.7%
Knife	0.267	0.817	+206%	0.229	0.886	+287%
Person	0.197	0.609	+209%	0.240	0.647	+170%
Scissors	0.761	0.929	+22.1%	0.760	0.967	+27.2%
All	0.472	0.801	+69.7%	0.465	0.847	+82.2%

TABLE 8. Per-Class TP/FN and Detection Recall (v5 Test Set)

Class	TP	FN	Recall
Hammer	~39	~7	0.848
Knife	~130	~29	0.817
Person	~807	~517	0.609
Scissors	~118	~9	0.929
Overall	~1,094	~562	0.801

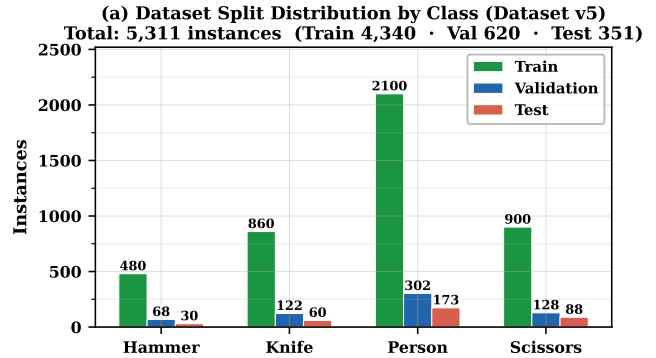
**FIGURE 20. Per-class precision-recall curves for the deployed v5 detector on the held-out test set.****TABLE 9. Dataset Versions: Growth and Test mAP**

Ver.	Train	Total	Inst.	mAP50	Primary change
v1	359	512	~1,200	0.465	Roboflow seed only
v2	1,383	1,730	~4,000	0.754	+ OpenImages v7 (1,219 imgs)
v5	4,982	6,226	16,335	0.847	+ OI per-class (2,276 imgs)

training set allows v5 to substantially outperform earlier checkpoints on every class. The class distribution across the train/val/test splits is plotted in Fig. 21, and the dataset growth across versions is summarised in Table 9.

I. MOBILENETV2 CLASSIFIER SPLIT

The MobileNetV2 *Safe/Weapon* classifier is trained on cropped regions harvested from the detection pipeline. The split is summarised in Table 10.

**FIGURE 21. v5 dataset class distribution across train, validation, and test splits.****TABLE 10. MobileNetV2 Classifier Dataset**

Split	Safe crops	Weapon crops	Total
Train	1,200	1,660	2,860
Val	200	24	224

J. HARDWARE INFERENCE PERFORMANCE

The custom *best_v5.hef* model achieves 52.2 FPS at 18.4 ms NPU latency on the Hailo-8L, comfortably exceeding the real-time requirement (≥ 25 FPS) for security surveillance. This compares favourably against the reference Hailo-distributed YOLOv8s (13.2 ms / 58.2 FPS); the modest 5 ms latency increase is attributable to the custom NMS post-processing baked into the HEF. Even with the small FPS delta, we remain well above the 30 FPS operating threshold and under the 50 ms latency ceiling, so the difference has no practical effect at runtime. The reference models run marginally faster because they were pre-optimised by the Hailo team for the exact fixed post-processing chain shipped in HailoRT. Fig. 22 visualises the latency/throughput comparison. Compared with prior edge-deployed detectors, our 52.2 FPS exceeds the 36.7 FPS reported by Xu *et al.* [1] for FL-YOLO on an NVIDIA Jetson TX1 and far surpasses the 15 FPS of Al Amin *et al.*'s FPGA implementation [5], while operating at a comparable mAP@0.5 of 0.847 on our domain-specific four-class task.

K. F1-CONFIDENCE THRESHOLD CURVES

All four classes reach their peak F1 score in the confidence range 0.25–0.27, which aligns with YOLO's default threshold of 0.25 and confirms that no manual threshold tuning is required for deployment. *Scissors* achieves the highest peak F1 (0.690), while *Person* is lowest (0.473), consistent with its

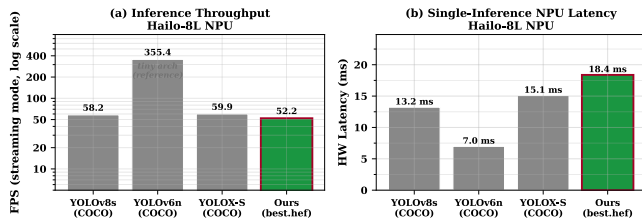


FIGURE 22. Hardware inference performance on the Hailo-8L: custom v5 HEF (52.2 FPS, 18.4 ms) versus the Hailo-provided YOLOv8s reference (58.2 FPS, 13.2 ms).

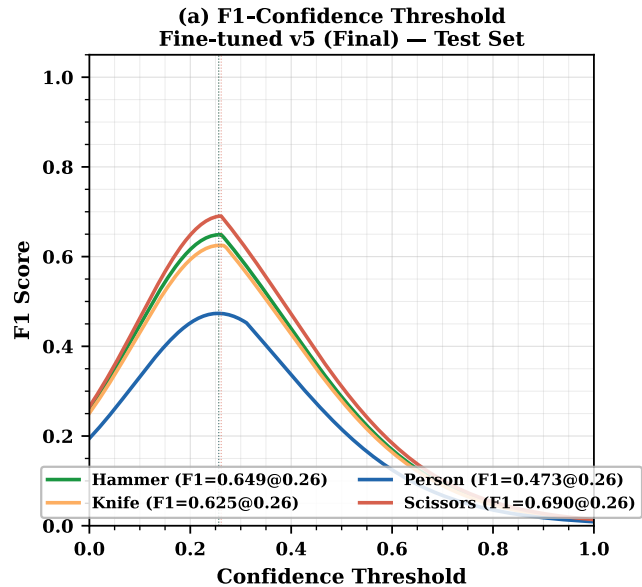


FIGURE 23. Per-class F1 score versus confidence threshold; peak F1 sits in the 0.25–0.27 band for all four classes.

lower recall; operating slightly below the peak threshold for *Person* can recover additional true positives at a modest false-positive cost. The F1-confidence sweep is shown in Fig. 23.

L. EFFECT OF TRAINING DATASET SIZE

Fine-tuned v1 (359 images) converges quickly to a high validation mAP of ~ 0.88 , but its held-out test mAP is only 0.465—exposing severe overfitting to the Roboflow dataset’s visual style. Fine-tuned v2 (1,383 images) narrows the gap substantially (val 0.735, test 0.754), and the final v5 (4,982 images, 150 epochs) achieves both strong validation (0.817) and the best test mAP (0.847). This confirms that dataset scale and diversity—not architectural changes—are the dominant factors for real-world generalisation on this task. The effect is plotted in Fig. 24.

M. VALIDATION-TO-TEST GENERALISATION GAP

Fine-tuned v1 exhibits a 0.414-point gap between its best validation mAP (0.879) and its held-out test mAP (0.465)—a clear sign of distribution overfit to the single-source Roboflow dataset. Fine-tuned v2 slightly reverses this gap (+0.019, test marginally beats val), and the final v5 achieves a

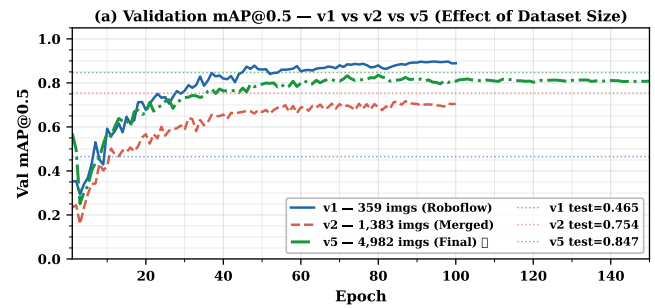


FIGURE 24. Validation mAP@0.5 progression versus training dataset size (v1, v2, v5).

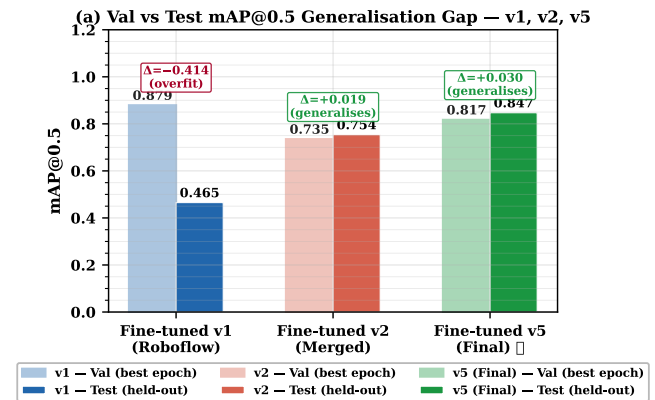


FIGURE 25. Validation-to-test mAP@0.5 gap across v1, v2, and v5. The final v5 achieves positive generalisation (test > val).

positive generalisation gap (+0.030: test 0.847 > val 0.817), confirming that the expanded 4,982-image dataset produces a model that generalises robustly to genuinely unseen real-world imagery. Fig. 25 visualises this progression.

VI. CONCLUSION

Project Garuda demonstrates that a self-hosted, privacy-first, cloud-independent AI home security system can be built from consumer hardware for under US\$500 without compromising on real-time performance or modern security primitives. The core engineering claim of the system is that a Raspberry Pi 5 paired with a Hailo-8L AI HAT+ can host the full inference stack—a custom-fine-tuned YOLOv8s detector for four threat-relevant classes, a MobileNetV2 classifier for *Safe/Weapon* verification, and a MiDaS Small depth estimator for monocular anti-spoof—at 52.22 FPS and 18.38 ms NPU latency, well inside the real-time envelope for continuous surveillance, while leaving the CPU free to run the FastAPI web server, the Narada voice assistant, the presence monitor, the deadman watchdog, and the logging engine concurrently.

On the model side, the progression from a 359-image v1 ($mAP_{50} = 0.465$) to the 4,982-image v5 ($mAP_{50} = 0.847$) with a +0.030 positive validation-to-test generalisation gap confirms that dataset scale and diversity—not architectural changes—are the dominant lever for robust real-world performance, and that the COCO baseline of 0.0002 mAP on

this 4-class task is essentially random, reinforcing the need for targeted fine-tuning on security-relevant classes. The low off-diagonal values in the confusion matrix (≤ 0.03) and the alignment of the peak F1 confidence with YOLO's default 0.25 threshold mean the model ships without requiring any post-deployment threshold tuning.

On the systems side, the multi-engine architecture—alert fan-out with 60-second SMTP cooldown, WebSocket push, AES-256-CBC clip encryption, and SSH exfiltration; PBKDF2-SHA256 with 600,000 iterations and OTP 2FA; the 180-second deadman watchdog; six lock-guarded and scheduler-driven operating modes; ARP-based LAN presence; and the SQLite-backed offline event sync—closes the gap identified in the introduction between DIY motion-only setups and cloud-dependent professional systems. Every event reaches the dashboard in real time over WebSocket or WebRTC, and every event is also persisted to the multi-tier logger for offline catch-up, giving browser clients a coherent view of the system state even after arbitrary network outages.

The broader implication is that edge AI is no longer a compromise. With the right accelerator, the right post-training quantisation flow, and a disciplined software architecture, a device the size of a paperback can deliver detection quality, response latency, and security properties that previously required a cloud subscription and a server rack. Future extensions worth pursuing include expanding the class vocabulary beyond the four threat-relevant labels, exploring temporal models for activity recognition on top of the detector, integrating on-device TTS so that Narada remains functional when the public network is unavailable, and replacing the current static MiDaS variance threshold with a learned liveness head that is co-trained with the detector.

REFERENCES

- [1] Z. Xu, J. Li, and M. Zhang, "A surveillance video real-time analysis system based on edge-cloud and FL-YOLO cooperation in coal mine," *IEEE Access*, vol. 9, pp. 161 498–161 512, 2021.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.
- [3] Hailo Technologies Ltd., "Hailo-8L M.2 AI acceleration module datasheet," 2024. [Online]. Available: <https://hailo.ai/products/ai-accelerators/hailo-8l-ai-accel-for-ai-light-applications/>
- [4] G. Bueno *et al.*, "Real-time edge computing vs. GPU-accelerated pipelines for low-cost microscopy applications," *Electronics*, vol. 14, no. 6, Art. no. 930, 2025.
- [5] R. Al Amin, M. Hasan, V. Wiese, and R. Obermaier, "FPGA-based real-time object detection and classification system using YOLO for edge computing," *IEEE Access*, vol. 12, pp. 61 080–61 093, 2024.
- [6] C. Dong, C. Pang, Z. Li, X. Zeng, and X. Hu, "PG-YOLO: A novel lightweight object detection method for edge devices in industrial Internet of Things," *IEEE Access*, vol. 10, pp. 40 177–40 186, 2022.
- [7] M.-A. Chung *et al.*, "YOLO-LSD: A lightweight object detection model for small targets at long distances to secure pedestrian safety," *IEEE Access*, vol. 13, pp. 14 520–14 534, 2025.
- [8] F. Cheng, "Accurate detection of solar panel defects using RMN-YOLO with multi-scale edge and attention enhancements," *IEEE Access*, vol. 13, pp. 25 632–25 645, 2025.
- [9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [10] Hailo Technologies Ltd., "TAPPAS — Hailo's AI application development framework," 2024. [Online]. Available: <https://github.com/hailo-ai/tappas>
- [11] Raspberry Pi Ltd., "Raspberry Pi 5 product brief," 2023. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-5/>
- [12] S. Ramírez, "FastAPI — Modern, fast web framework for building APIs with Python," 2019. [Online]. Available: <https://fastapi.tiangolo.com>
- [13] GStreamer Team, "GStreamer: Open source multimedia framework," 2024. [Online]. Available: <https://gstreamer.freedesktop.org>
- [14] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.
- [15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.
- [16] B. Kaliski, "PKCS #5: Password-based cryptography specification version 2.0," RFC 2898, Internet Eng. Task Force, Sep. 2000.
- [17] Cloudflare, Inc., "Cloudflare Tunnel documentation," 2024. [Online]. Available: <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>

...